

Architecting the Self-Evolving Knowledge Base: A System Design for Agentic Research Workflows in TiddlyWiki

1. Introduction: The Paradigm Shift to Generative Knowledge Management

The contemporary landscape of Personal Knowledge Management (PKM) is undergoing a radical transformation, driven by the integration of Large Language Models (LLMs) into the very fabric of information storage systems. Historically, tools like TiddlyWiki served as passive repositories—digital gardens where the gardener was solely responsible for seeding, watering, and harvesting ideas. However, the emergence of agentic workflows¹ allows for a fundamental architectural shift: the transition from a static "Second Brain" to an active, generative "Wisdom Engine".³ This report details the comprehensive design of an autonomous research system built upon TiddlyWiki's Node.js configuration, designed to execute complex cognitive tasks ranging from deep literature review to automated scientific hypothesis generation and final publication in LaTeX.

The proposed architecture addresses the limitations of traditional Retrieval-Augmented Generation (RAG), which often suffers from "context poisoning" and an inability to reason over complex, multi-hop relationships.⁴ By leveraging TiddlyWiki's inherent graph structure—composed of atomic units known as "tiddlers"—and combining it with agentic reasoning loops, the system enables a recursive process of knowledge discovery. This system does not merely store information; it actively identifies gaps, resolves contradictions, and synthesizes new insights, effectively mimicking the collaborative dynamics of a human research team within a single software instance.⁶

Utilizing the user's existing infrastructure—a PDF-to-TiddlyWiki conversion pipeline, a categorical tagging system, and a LaTeX publication workflow—we establish a set of autonomous agents triggered by state changes in the knowledge base. These agents,

operating via the TiddlyWiki WebServer API and file-system watchers⁷, execute tasks that propel a research query from a raw idea to a polished, mathematically rigorous publication. The following analysis exhaustively explores the theoretical underpinnings, technical implementation details, and token-optimization strategies required to realize this vision of Agentic TiddlyWiki.

2. Theoretical Framework: The Agentic Zettelkasten

2.1 Atomicity and Emergent Intelligence

The foundation of this architecture lies in the Zettelkasten method, which prioritizes the "atomicity" of notes—ensuring each concept is self-contained and distinct.⁹ In the context of LLM agents, atomicity is not merely an organizational preference but a computational necessity. Large Language Models struggle with massive, unstructured contexts; they perform optimally when information is segmented into discrete, semantic units that can be dynamically retrieved and reassembled.¹⁰

TiddlyWiki's architecture, where every paragraph, definition, or formula can be its own tiddler, aligns perfectly with this requirement. By treating the tiddler as the fundamental unit of computation, agents can engage in "VectorRAG," where each atomic note is embedded into a vector space for semantic retrieval, and "GraphRAG," where the explicit links between tiddlers facilitate structural reasoning.¹¹ This dual approach allows the system to transcend simple keyword matching. For instance, an agent investigating "Agentic RAG" can retrieve not just documents containing that phrase, but also tiddlers linked to "Multi-step reasoning" or "Tool use," identifying conceptual connections that mere vector similarity might miss.⁹

2.2 Overcoming the Collector's Fallacy

A pervasive issue in PKM is the "Collector's Fallacy"—the erroneous belief that merely collecting references constitutes knowledge acquisition.¹² In traditional workflows, researchers amass PDFs and URLs which often remain unread or unintegrated. The proposed Agentic TiddlyWiki actively combats this by assigning the role of "active processor" to the AI

agents.

When a new document is ingested, it is not simply archived. An agent, triggered by the ingestion event, immediately decomposes the document into its constituent claims, evidence, and definitions. This process transforms passive storage into active "context engineering," where the system continuously reorganizes its internal graph to reflect new information.¹³ The agents serve as a cognitive adversarial force, constantly challenging the user's existing notes with new data, thereby preventing the stagnation of the knowledge base. This dynamic evolution distinguishes an agentic system, which maintains control over task execution and tool usage, from a simple automation script.¹

2.3 Taxonomy of Research Agents

To manage the complexity of the research lifecycle, we define a taxonomy of specialized agents. These are not distinct software programs, but rather specific "personas" or prompting strategies invoked by the central control loop based on TiddlyWiki tags.

Agent Persona	Trigger Criteria	Cognitive Function	Operational Output
The Librarian	Tag: todo + Literature Review	Retrieval, filtering, and ingestion.	Atomic Source Tiddlers, Annotated Bibliographies.
The Analyst	Tag: todo + Deep Research	Pattern recognition, gap analysis, graph traversal.	Insight Tiddlers, Conflict Resolution Notes.
The Scientist	Tag: todo + Hypothesis Generation	Abductive reasoning, experimental design.	Hypothesis Tiddlers, Experimental Proposals.
The Scribe	Tag: todo + Draft Publication	Synthesis, narrative construction, formatting.	Master Publication Tiddlers, LaTeX/Pandoc Exports.

The Critic	Tag: status:review	Adversarial review, logical consistency checking.	Critique Tiddlers, "Needs Revision" Flags.
-------------------	--------------------	---	--

3. Technical Architecture: The Node.js Ecosystem

3.1 File System vs. API Interaction Models

Implementing this architecture requires the Node.js version of TiddlyWiki, which stores tiddlers as individual text files (typically .tid) on the host file system.¹⁴ This configuration offers two distinct pathways for agent interaction, each serving different stages of the workflow.

The first pathway is **File System Watchers**. External scripts (Python or Node.js) monitor the tiddlers/ directory for changes. When a user creates a tiddler with the tag todo, the watcher detects the file creation event and triggers the appropriate agent.⁷ This method is highly efficient for batch processing and heavy computational tasks, such as generating vector embeddings for thousands of tiddlers, as it bypasses the HTTP overhead. It allows agents to read the raw text files directly, processing the JSON-formatted metadata and Wikitext content without rendering.¹⁷

The second pathway is the **TiddlyWiki WebServer API**. For operations that require immediate feedback within the user interface or need to respect the internal logic of the running kernel (such as updating system tiddlers or triggering other plugins), agents utilize HTTP requests. The PUT /recipes/default/tiddlers/{title} endpoint allows agents to inject new tiddlers or update existing ones programmatically.⁸ This is crucial for tasks where the agent acts as a "Scribe," creating drafts that the user might be editing simultaneously. The API ensures that concurrent edits do not corrupt the data integrity, a risk present when modifying files directly on the disk while the server is running.¹⁵

3.2 The Dual-Layer Token Format

A significant constraint in LLM-based research is the context window—the limit on how much text an agent can process at once. To address this, the system assumes a "token optimized format." In TiddlyWiki, this translates to a dual-layer data representation:

1. **The Structural Layer:** This is the native TiddlyWiki format, relying heavily on transclusions (e.g., `{{Definition:Entropy}}`). This layer is "token-light" because it contains pointers rather than full text. It is used for organizational tasks, outlining, and structural editing.
2. **The Semantic Layer:** For tasks requiring deep understanding (like hypothesis generation), the system generates a "flattened" version of the text where transclusions are recursively resolved.¹⁹

However, passing the fully flattened text of a large knowledge base to an LLM is prohibitively expensive and often exceeds token limits. Therefore, the architecture implements a **Context Compression Strategy**. Instead of flattening the entire graph, the system provides the agent with a "linearized" subgraph.²¹ This involves selecting the central tiddler (the research topic) and "walking" the graph to retrieve only the immediate neighbors (definitions, direct evidence). This subgraph is then serialized into a JSON schema or a compact text format that defines the entities and their relationships, allowing the agent to reason about the structure without consuming tokens on irrelevant details.²³

3.3 Handling External Assets and Images

Research often involves visual data. In the Node.js configuration, TiddlyWiki supports external images, which are stored as standard image files (JPG, PNG) in a subdirectory, while a corresponding `_canonical_uri` tiddler acts as a pointer.²⁵ This is essential for the publication pipeline. Agents generating LaTeX drafts must ensure that image references point to these local files rather than trying to embed base64 data, which would break the LaTeX compiler.²⁶

The agents are programmed to recognize the `[img[...]]` syntax and cross-reference it with the file system. If an agent retrieves an external paper containing figures, it extracts these images, saves them to the `images/` directory, and creates the appropriate `_canonical_uri` tiddler, ensuring the visual evidence is available for both the TiddlyWiki user interface and the eventual PDF export.²⁸

4. Operational Workflow 1: Deep Research and Knowledge Acquisition

4.1 Triggering the Librarian Agent

The workflow begins when the user creates a tiddler such as Research Query: Agentic RAG Scalability, tagged with `todo` and `category:Deep Research`. The *Librarian* agent detects this tag and initiates the Deep Research protocol.

4.2 Ontology Expansion and Internal Graph Traversal

Before accessing the internet, the agent performs an internal "sanity check" to ground the query in the user's existing knowledge. This utilizes a **GraphRAG** approach. The agent does not merely search for keywords; it traverses the link structure of the wiki.⁵

1. **Entity Extraction:** The agent parses the query to identify core entities (e.g., "Agentic RAG," "Scalability").
2. **Recursive Traversal:** It identifies tiddlers that link to these entities. If the user has a note on "Vector Databases" that links to "Scalability," the agent pulls this into the context.³¹
3. **Gap Identification:** The agent compares the retrieved internal context against the complexity of the query. If the internal graph lacks nodes related to "Multi-agent coordination" (a key component of Agentic RAG), it marks this as a knowledge gap.³²

This step is crucial for preventing "hallucinations" where the AI invents facts that contradict the user's established definitions. By grounding the research in the local graph, the agent ensures consistency with previous work.³

4.3 External Search and Atomic Ingestion

To fill the identified gaps, the Librarian queries external academic repositories (arXiv, PubMed, Semantic Scholar). Upon retrieving relevant papers, it does not dump the full text into a single tiddler. Instead, it applies **Semantic Chunking** and **Zettelkasten Atomicity**.¹⁰

The agent decomposes a paper into multiple tiddlers:

- **Source Tiddler:** Contains the BibTeX citation data, abstract, and a link to the PDF.
- **Concept Tiddlers:** For every new term defined in the paper (e.g., "Chain-of-Thought"), a separate definition tiddler is created.
- **Claim Tiddlers:** Key findings are extracted as individual tiddlers (e.g., "Claim: CoT improves reasoning in RAG"). These are linked back to the Source Tiddler and tagged with category:Claim.⁹

This atomization allows future agents to cite specific claims without loading the entire paper, significantly optimizing token usage.³⁵ The agent utilizes a "JSON Schema" output format to ensure that the extracted data strictly adheres to the TiddlyWiki field structure (title, text, tags, fields).²⁴

4.4 The Analyst: Synthesis and Graph Construction

Once the new data is ingested, the *Analyst* agent activates. Its role is to weave the new "Atomic Tiddlers" into the existing knowledge graph.

1. **Link Prediction:** The Analyst scans the new tiddlers for keywords that match existing tiddler titles. It automatically inserts links (e.g., adding [[Zettelkasten]] to a new note about "Atomic Notes").
2. **Conflict Detection:** The agent compares new claims against existing ones. If the new paper states "RAG reduces hallucination" but an existing note argues "RAG induces context poisoning," the Analyst creates a **Conflict Tiddler** highlighting this discrepancy.³⁷
3. **Insight Generation:** The agent synthesizes multiple atomic tiddlers into an **Insight Tiddler**—a higher-level summary that connects the dots. This tiddler uses transclusions ({{Claim A}} supports {{Claim B}}) to build a narrative structure while maintaining data lineage.³⁴

5. Operational Workflow 2: Hypothesis Generation and Scientific Discovery

5.1 The AI Co-Scientist Model

Moving beyond synthesis, the system engages in active scientific discovery. Triggered by category:Hypothesis Generation, the *Scientist* agent utilizes the "AI Co-Scientist" framework.⁶ This involves multi-agent collaboration where different personas (the Generator and the Skeptic) debate to refine ideas.³⁹

5.2 Graph Linearization and Contextual Reasoning

To generate high-quality hypotheses, the agent requires a "World Model" of the research topic. Since LLMs cannot "see" the graph visually, the system performs **Graph Linearization**.²² It converts the relevant subgraph into a sequence of logical assertions:

- *Input*: Node A (Agent) -> Edge (uses) -> Node B (Tools).
- *Linearized Context*: "Agents use Tools. Tools require Interfaces. Interfaces define Protocols."

This linearized context allows the LLM to perform multi-hop reasoning. The *Scientist* agent identifies paths in the graph that are logically plausible but currently unconnected—representing potential research hypotheses.⁴⁰

5.3 Adversarial Refinement Loop

The generation process is iterative. The *Scientist* proposes a hypothesis (e.g., "Dynamic Graph Linearization optimizes Token Usage in RAG"). The *Critic* agent then attacks this hypothesis, searching the knowledge base for counter-evidence (e.g., "Linearization destroys local topology info").⁴²

This adversarial loop continues until a robust hypothesis is formed. The result is saved as a **Hypothesis Tiddler**, structured with sections for "Proposed Mechanism," "Expected Outcome," and "Experimental Validation".⁴³ This automated reasoning process shifts the burden of initial ideation from the human researcher to the agentic system, allowing the user to focus on validation and high-level direction.⁴⁴

6. Operational Workflow 3: Automated Publication Pipeline

6.1 The Scribe: Structural Outlining and Drafting

The final phase is the production of a publication. The *Scribe* agent is triggered by a tiddler titled "Title for new publication about XYZ" with tags `todo` and `Draft Publication`.

The Scribe first generates a "Master Tiddler" that serves as the document skeleton. It uses a template consistent with the target publication venue (e.g., IEEE, ACM, arXiv).³⁴ This master tiddler does not contain the full text; rather, it contains a sequence of transclusions:

Code snippet

```
! Introduction
{{Draft:XYZ/Introduction}}
```

```
! Related Work
{{Draft:XYZ/Related_Work}}
```

```
! Methodology
{{Draft:XYZ/Methodology}}
```

This modular approach allows for non-linear editing. The agent can update the "Methodology" section without locking the "Introduction" file.

6.2 Transclusion Management and Token Economy

When writing the content, the Scribe utilizes the **Structural Layer** of the token-optimized format. It creates sentences that mix narrative text with transcluded definitions:

"As established in {{Definition:GraphRAG}}, the primary advantage is..."

This ensures that if the user later refines the definition of GraphRAG, the publication automatically reflects the most current understanding. However, this introduces the risk of **Recursive Transclusion Errors**.⁴⁵ If Tiddler A transcludes Tiddler B, and Tiddler B transcludes Tiddler A, the rendering engine will crash.

To mitigate this, the Scribe runs a **Dependency Check** script (via the Analyst persona) before finalizing any draft. This script traverses the transclusion tree to detect cycles and flatten them where necessary.⁴⁶

6.3 The LaTeX/Pandoc Bridge

The system assumes a complete process for generating LaTeX from TiddlyWiki. This is achieved via **Pandoc**, a universal document converter.⁴⁷

The Scribe creates a tiddlywiki.files configuration that exports the Master Tiddler and its dependencies as a single static HTML file or a flat Markdown file. This file is then passed to Pandoc.

- **Pandoc Filters:** Custom filters (written in Python or Lua) are employed to translate TiddlyWiki-specific syntax into LaTeX. For example, a filter detects the {{...}} transclusion syntax and converts it into LaTeX \input{...} commands or simply embeds the text content.²⁸
- **Bibliography Handling:** The Scribe scans the draft for citation keys (e.g., [@smith2023]). It retrieves the BibTeX data from the corresponding Source Tiddlers and generates a .bib file, which Pandoc uses to compile the references.³⁴
- **Image Handling:** The Scribe ensures that all images included via [img[...]] are present in the export directory. It modifies the image paths in the LaTeX source to be relative to the compilation root, preventing "File Not Found" errors during the final PDF build.²⁹

7. Token Optimization Strategy: The Context Economy

The viability of this entire architecture rests on efficient token management. Agents cannot simply "read the whole wiki." We implement specific strategies to optimize the Context Economy.

7.1 Summarization and Hierarchical Retrieval

For broad tasks like "Literature Review," the agents do not read full papers. They rely on **Hierarchical Retrieval**.

1. **Level 1 (Metadata):** The agent scans titles and tags.
2. **Level 2 (Abstracts):** If relevant, it reads the abstract/summary tiddler.
3. **Level 3 (Full Text):** Only for the most critical claims does it access the full text.⁴⁹

This is supported by "Insight Tiddlers" which are pre-summarized chunks of knowledge. The agent prefers to read one Insight Tiddler rather than the ten Source Tiddlers it summarizes.³⁵

7.2 The Definition Injection Pattern

When an agent needs to understand a specific tiddler that is heavy with transclusions, we use the Definition Injection Pattern.

Instead of forcing the LLM to request each transcluded tiddler individually (which incurs high latency and token costs for API calls), the system pre-fetches the text of all transcluded tiddlers and injects them into the prompt context as a "Glossary."

Prompt Structure:

SYSTEM: You are a research assistant.

GLOSSARY:

: "Graph Retrieval Augmented Generation..."

: "Token usage scales quadratically with context length..."

TASK: Analyze the following text:

"The implementation of {{Definition:GraphRAG}} is limited by {{Claim:TokenCost}}."

This allows the agent to "resolve" the transclusions mentally without requiring the text to be physically flattened in the source file, preserving the graph structure.¹⁹

8. Comparison: Baseline RAG vs. Agentic TiddlyWiki

To underscore the value of this architecture, we compare it against a standard RAG implementation.

Feature	Baseline RAG	Agentic TiddlyWiki
Retrieval Mechanism	Vector Similarity Search (Semantic)	Hybrid: Vector + Graph Traversal + Link Following
Context Structure	Unstructured chunks of text	Atomic, structured Tiddlers with metadata
Reasoning Capability	Single-step retrieval and generation	Multi-hop reasoning via Graph Linearization
Knowledge Evolution	Static (Read-only)	Generative (Agents create/update knowledge)
Handling Contradictions	Often ignores or hallucinates resolution	Explicitly identifies and flags conflicts
Output Format	Chat response	Structured, citation-backed LaTeX publication

The Baseline RAG approach struggles with the complex, interconnected nature of academic research.² It retrieves paragraphs that "look" similar but fails to understand the causal chain of arguments. The Agentic TiddlyWiki, by mirroring the Zettelkasten structure, preserves these causal links, allowing the *Analyst* and *Scientist* agents to perform true "Deep Research" rather than surface-level summarization.⁵²

9. Implementation Roadmap

The deployment of this system follows a phased approach, establishing the foundation before enabling full autonomy.

Phase 1: Infrastructure and Ontology

- **Action:** Deploy Node.js TiddlyWiki instance.

- **Action:** Define the TiddlyWiki tagging ontology (Category, Status, Priority).
- **Action:** Set up the Python/Node.js file watchers to monitor the tiddlers/ directory.

Phase 2: The Librarian (Ingestion)

- **Action:** Implement the API connection to arXiv/Semantic Scholar.
- **Action:** Develop the PDF-to-Tiddler conversion script with Zettelkasten atomization logic.
- **Action:** Train/Prompt the Librarian agent to extract "Claims" and "Definitions" reliably.

Phase 3: The Analyst (Graph Building)

- **Action:** Implement the GraphRAG traversal algorithm.
- **Action:** Develop the "Conflict Detection" prompt strategies.
- **Action:** Enable the agent to write to the file system via the WebServer API (PUT requests).

Phase 4: The Scientist (Discovery)

- **Action:** Integrate the "AI Co-Scientist" multi-agent workflow.
- **Action:** Implement Graph Linearization for hypothesis generation contexts.
- **Action:** Establish the adversarial "Critic" loop for hypothesis refinement.

Phase 5: The Scribe (Publication)

- **Action:** Create the Master Publication templates.
- **Action:** Configure the Pandoc filters for TiddlyWiki syntax.
- **Action:** Automate the final LaTeX compilation and .bib generation.

10. Conclusion

The architecture proposed in this report represents a significant leap forward in the domain of Personal Knowledge Management. By coupling the flexible, atomic structure of TiddlyWiki with the generative capabilities of agentic AI, we create a system that is not merely a storage device, but a collaborator. The separation of concerns—using Node.js for robust file management, the TiddlyWiki graph for structural reasoning, and autonomous agents for cognitive processing—creates a resilient and scalable workflow.

This system solves the fundamental challenge of the "Information Age": the gap between information access and knowledge creation. Through the coordinated efforts of the Librarian, Analyst, Scientist, and Scribe agents, the user is freed from the drudgery of data management and empowered to focus on high-level synthesis and direction. The TiddlyWiki instance becomes a living, breathing entity, constantly evolving, checking itself for consistency, and actively propelling the user toward new scientific discoveries and publications. This is the realization of the "Wisdom Engine," turning the static archive into a dynamic engine of intellectual production.³

11. Detailed Agent Prompt Specifications

To ensure the practical applicability of this report, we provide detailed specifications for the core prompts driving the agents.

11.1 The Librarian Prompt (Ingestion)

ROLE: You are an expert Academic Librarian and Ontologist.

INPUT: Raw text of a research paper.

TASK: Decompose this paper into atomic TiddlyWiki tiddlers.

CONSTRAINTS:

1. Create a 'Source' tiddler with BibTeX fields.
2. Extract every unique definition into a separate tiddler titled "Definition:".
3. Extract key claims into tiddlers titled "Claim:".
4. Ensure all Claims link back to the Source tiddler using the field `source_ref`.
5. Output MUST be a JSON array of tiddler objects.

11.2 The Analyst Prompt (GraphRAG)

ROLE: You are a Senior Research Analyst.

INPUT: A JSON object representing a subgraph of tiddlers (Nodes and Links).

TASK: Identify relationships and contradictions.

STEPS:

1. Analyze the connections between 'Claim' tiddlers.
2. If Claim A (from Source X) contradicts Claim B (from Source Y), create a tiddler titled "Conflict:".
3. If multiple claims support a broader trend, create an "Insight:" tiddler.
4. Use transclusions {{...}} to reference the original claims in your text.

11.3 The Scribe Prompt (Drafting)

ROLE: You are an Academic Writer specializing in Computer Science.

INPUT: An outline tiddler and a list of available context definitions.

TASK: Write the content for the section "{{Draft:XYZ/Methodology}}".

REQUIREMENTS:

1. Write in formal academic prose.
2. Do NOT define terms. Instead, transclude the definition: "We utilize {{Definition:GraphRAG}} to..."
3. If you need to cite a source, use the format {{Citation:AuthorYear}}.
4. Incorporate the provided images using [img[...]] syntax where relevant.

This rigorous structured approach ensures that the agents act deterministically within the bounds of the system architecture, maintaining the integrity of the knowledge base while providing the flexibility required for creative research.

Works cited

1. Building Effective AI Agents - Anthropic, accessed November 19, 2025, <https://www.anthropic.com/research/building-effective-agents>
2. [2501.09136] Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG - arXiv, accessed November 19, 2025, <https://arxiv.org/abs/2501.09136>
3. Implementing a 'Wisdom Engine' for Personal Knowledge Management | by Sensay, accessed November 19, 2025, <https://asksensay.medium.com/implementing-a-wisdom-engine-for-personal-knowledge-management-3c76b8d8f760>
4. Knowledge Graph-based Retrieval-Augmented Generation for Schema Matching

- arXiv, accessed November 19, 2025, <https://arxiv.org/html/2501.08686v1>
- 5. Implementing Graph RAG Using Knowledge Graphs - IBM, accessed November 19, 2025, <https://www.ibm.com/think/tutorials/knowledge-graph-rag>
- 6. Accelerating scientific breakthroughs with an AI co-scientist - Google Research, accessed November 19, 2025, <https://research.google/blog/accelerating-scientific-breakthroughs-with-an-ai-co-scientist/>
- 7. TiddlyWiki Integration / Automation - Talk TW, accessed November 19, 2025, <https://talk.tiddlywiki.org/t/tiddlywiki-integration-automation/12924?page=2>
- 8. WebServer API for PUT a file - Talk @ TiddlyWiki, accessed November 19, 2025, <https://talk.tiddlywiki.org/t/webserver-api-for-put-a-file/3601>
- 9. A-Mem: Agentic Memory for LLM Agents - arXiv, accessed November 19, 2025, <https://arxiv.org/html/2502.12110v1>
- 10. Optimizing RAG Context: Chunking and Summarization for Technical Docs, accessed November 19, 2025, <https://dev.to/oleh-halytskyi/optimizing-rag-context-chunking-and-summarization-for-technical-docs-3pel>
- 11. Open-Source Agentic Hybrid RAG Framework for Scientific Literature Review - arXiv, accessed November 19, 2025, <https://arxiv.org/html/2508.05660v1>
- 12. How To Build Your Zettelkasten to Master AI, accessed November 19, 2025, <https://zettelkasten.de/posts/how-to-build-zettelkasten-master-ai/>
- 13. getzep/graphiti: Build Real-Time Knowledge Graphs for AI Agents - GitHub, accessed November 19, 2025, <https://github.com/getzep/graphiti>
- 14. Using TiddlyWiki on Node.js, accessed November 19, 2025, <https://tiddlywiki.com/static/Using%20TiddlyWiki%20on%20Node.js.html>
- 15. TiddlyWiki on Node.js, accessed November 19, 2025, <https://tiddlywiki.com/static/TiddlyWiki%20on%20Node.js.html>
- 16. How to run server-side javascript using Tiddlywiki node.js? : r/TiddlyWiki5 - Reddit, accessed November 19, 2025, https://www.reddit.com/r/TiddlyWiki5/comments/jks8xy/how_to_run_serverside_javascript_using_tiddlywiki/
- 17. Writing Shell Scripts Against a TiddlyWiki, accessed November 19, 2025, <https://groktiddlywiki.com/static/Writing%20Shell%20Scripts%20Against%20a%20TiddlyWiki.html>
- 18. WebServer API: Put Tiddler: TiddlyWiki v5.3.8 — a non-linear personal web notebook, accessed November 19, 2025, <https://tiddlywiki.com/static/WebServer%20API%253A%2520Put%2520Tiddler.html>
- 19. Filters and Transclusions - Grok TiddlyWiki, accessed November 19, 2025, <https://groktiddlywiki.com/static/Filters%20and%20Transclusions.html>
- 20. Transclusion: TiddlyWiki v5.3.8 — a non-linear personal web notebook, accessed November 19, 2025, <https://tiddlywiki.com/static/Transclusion%20in%20WikiText.html>
- 21. Graph RAG: Navigating graphs for Retrieval-Augmented Generation using Elasticsearch, accessed November 19, 2025,

- <https://www.elastic.co/search-labs/blog/rag-graph-traversal>
22. Graph Linearization Methods for Reasoning on Graphs with Large Language Models - arXiv, accessed November 19, 2025, <https://arxiv.org/html/2410.19494v3>
 23. Applications of Knowledge Graphs in LLMs: 3 Important Cases - Data Science Dojo, accessed November 19, 2025, <https://datasciencedojo.com/blog/knowledge-graphs/>
 24. neo4j_graphrag.generation.prompts — neo4j-graphrag-python documentation, accessed November 19, 2025, https://neo4j.com/docs/neo4j-graphrag-python/current/_modules/neo4j_graphrag/generation/prompts.html
 25. ExternallImages: TiddlyWiki v5.3.8 — a non-linear personal web notebook, accessed November 19, 2025, <https://tiddlywiki.com/static/ExternallImages.html>
 26. Use external images in node js - Talk @ TiddlyWiki, accessed November 19, 2025, <https://talk.tiddlywiki.org/t/use-external-images-in-node-js/3332>
 27. Export tiddler as static html with embedding external images - Talk @ TiddlyWiki, accessed November 19, 2025, <https://talk.tiddlywiki.org/t/export-tiddler-as-static-html-with-embedding-external-images/4341>
 28. Pandoc: Exporting Tikz Figures and Other Environments from Latex to other formats - TeX, accessed November 19, 2025, <https://tex.stackexchange.com/questions/445051/pandoc-exporting-tikz-figures-and-other-environments-from-latex-to-other-format>
 29. [TW5] How to export image files - Google Groups, accessed November 19, 2025, <https://groups.google.com/g/tiddlywiki/c/tcywN1Tnhhs>
 30. GraphRAG-powered AI Agent interfaces: Revolutionizing content retrieval and generation with graph-augmented intelligence (Part 1 of 2) | by Yasmin Bokobza | Data Science at Microsoft | Medium, accessed November 19, 2025, <https://medium.com/data-science-at-microsoft/graphrag-powered-ai-agent-interfaces-revolutionizing-content-retrieval-and-generation-with-8ba7d2c1c698>
 31. Advanced RAG Techniques for High-Performance LLM Applications - Graph Database & Analytics - Neo4j, accessed November 19, 2025, <https://neo4j.com/blog/genai/advanced-rag-techniques/>
 32. Mixture of Knowledge Minigraph Agents for Literature Review Generation - AAAI Publications, accessed November 19, 2025, <https://ojs.aaai.org/index.php/AAAI/article/view/34796/36951>
 33. Optimizing Text Input for RAG Models: Chunking & Splitting Strategies - Successive Digital, accessed November 19, 2025, <https://successive.tech/blog/rag-models-optimizing-text-input-chunking-splitting-strategies/>
 34. TwiddlyWiki for long form writing: plugins, workflows, etc? : r/TiddlyWiki5 - Reddit, accessed November 19, 2025, https://www.reddit.com/r/TiddlyWiki5/comments/gysx7h/twiddlywiki_for_long_form_writing_plugins/
 35. How to Reduce Massive Token Usage in a Multi-LLM Text-to-SQL RAG Pipeline? - Reddit, accessed November 19, 2025,

- https://www.reddit.com/r/Rag/comments/1ojnef5/how_to_reduce_massive_token_usage_in_a_multillm/
36. Implementing Retrieval Augmented Generation (RAG) with JSON Output Using Nuclia RAG as a Service, accessed November 19, 2025, <https://nuclia.com/developers/implementing-retrieval-augmented-generation-rag-with-json-output-using-nuclia-rag-as-a-service/>
 37. Automated literature research and review-generation method based on large language models - Oxford Academic, accessed November 19, 2025, <https://academic.oup.com/nsr/advance-article/doi/10.1093/nsr/nwaf169/8120226>
 38. Intelligent Tiddlywiki: Machine Learning ("AI") Enhancements - ai - Talk TW, accessed November 19, 2025, <https://talk.tiddlywiki.org/t/intelligent-tiddlywiki-machine-learning-ai-enhancements/11046>
 39. A Survey on Hypothesis Generation for Scientific Discovery in the Era of Large Language Models - arXiv, accessed November 19, 2025, <https://arxiv.org/html/2504.05496v1>
 40. Need a research hypothesis? Ask AI. | MIT News | Massachusetts Institute of Technology, accessed November 19, 2025, <https://news.mit.edu/2024/need-research-hypothesis-ask-ai-1219>
 41. Embracing Foundation Models for Advancing Scientific Discovery - PMC - PubMed Central, accessed November 19, 2025, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11923747/>
 42. Debate on Graph: a Flexible and Reliable Reasoning Framework for Large Language Models - arXiv, accessed November 19, 2025, <https://arxiv.org/html/2409.03155v1>
 43. Hypothesis generation project, accessed November 19, 2025, <https://chicagohai.github.io/hypogenic-demo/>
 44. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery - Sakana AI, accessed November 19, 2025, <https://sakana.ai/ai-scientist/>
 45. Transclusion Basic Usage - TiddlyWiki, accessed November 19, 2025, <https://tiddlywiki.com/static/Transclusion%2520Basic%2520Usage.html>
 46. How to use the transclude widget and shorthand transcludes - Talk @ TiddlyWiki, accessed November 19, 2025, <https://talk.tiddlywiki.org/t/how-to-use-the-transclude-widget-and-shorthand-transcludes/1163>
 47. Pandoc User's Guide, accessed November 19, 2025, <https://pandoc.org/MANUAL.html>
 48. Pandoc convert docx to markdown with embedded images - Stack Overflow, accessed November 19, 2025, <https://stackoverflow.com/questions/39956497/pandoc-convert-docx-to-markdown-with-embedded-images>
 49. Contextual Compression in Retrieval-Augmented Generation for Large Language Models: A Survey - arXiv, accessed November 19, 2025, <https://arxiv.org/html/2409.13385v1>
 50. Optimizing RAG. RAG Demystified: A Hands-On Guide to... | by Skanda Vivek |

EMAlpha, accessed November 19, 2025,

<https://medium.com/emalpha/optimizing-rag-bd65ebc5e51a>

51. How to Build a RAG Workflow for Agentic AI without Code | Yugabyte, accessed November 19, 2025,

<https://www.yugabyte.com/blog/build-a-rag-workflow-for-agentic-ai-without-code/>

52. Welcome - GraphRAG, accessed November 19, 2025,

<https://microsoft.github.io/graphrag/>